

12. Algoritmos de ordenamiento

Ordenar es reacomodar los elementos de una colección (como una lista) según algún criterio. En programación, el criterio más común es de **menor a mayor** o **mayor a menor**.

¿Por qué es importante el ordenamiento?

El ordenamiento es una operación básica pero fundamental en la informática, ya que:

- Facilita la **búsqueda** eficiente (como en la búsqueda binaria).
- Permite **organizar datos** para visualizaciones o reportes.
- Es la base de muchos algoritmos y estructuras más avanzadas.

Tipos de ordenamiento

Hay múltiples algoritmos de ordenamiento, cada uno con sus ventajas y desventajas. Algunos son más simples y didácticos, otros son extremadamente eficientes en contextos reales. Los que trabajaremos son:

1. **Bubble Sort mejorado**
Ordena intercambiando pares de elementos adyacentes. Versión mejorada: si no hubo cambios en una pasada, se detiene.
2. **Selection Sort**
Busca el mínimo y lo coloca en la primera posición, repitiendo el proceso para el resto de la lista.
3. **Insertion Sort**
Construye la lista ordenada elemento por elemento, insertando cada nuevo elemento en el lugar correcto.
4. **Merge Sort**
Divide recursivamente la lista en mitades, las ordena por separado y luego las fusiona de forma ordenada.
5. **Quick Sort**
Elige un “pivote”, reorganiza los elementos menores a la izquierda y mayores a la derecha, y aplica el proceso recursivamente.

¿Qué pasa en memoria?

Todos estos algoritmos trabajan con listas que pueden ser modificadas **en el lugar** (in-place), o creando nuevas estructuras auxiliares. Algunos como Merge Sort requieren más espacio en memoria porque dividen y fusionan listas nuevas. Otros como Selection o Insertion pueden funcionar sin estructuras adicionales.

También es importante notar cuántas comparaciones y desplazamientos realiza cada algoritmo, ya que eso impacta directamente en el **rendimiento**, especialmente para listas grandes.

¿Por qué no usar directamente `list.sort()`?

Python ofrece métodos muy optimizados como `list.sort()` o `sorted()`, pero **es fundamental comprender los principios detrás de estos métodos**. Entender cómo ordena una lista desde cero permite:

- Tomar decisiones correctas en algoritmos propios.
- Elegir el mejor método según el contexto.
- Evitar errores comunes y optimizar programas.

12.1. Bubble Sort Mejorado

Bubble Sort compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Este proceso se repite varias veces, y en cada pasada el elemento más grande "burbujea" hacia el final.

La **versión mejorada** introduce una optimización:

Si en una pasada completa no se hicieron intercambios, eso significa que la lista ya está ordenada y no hace falta seguir iterando.

Paso a paso

Supongamos la lista:

[5, 2, 4, 1]

1. Primera pasada:
 - Compara 5 y 2 → intercambia → **[2, 5, 4, 1]**
 - Compara 5 y 4 → intercambia → **[2, 4, 5, 1]**
 - Compara 5 y 1 → intercambia → **[2, 4, 1, 5]**
2. Segunda pasada:
 - Compara 2 y 4 → ok
 - Compara 4 y 1 → intercambia → **[2, 1, 4, 5]**
 - Compara 4 y 5 → ok
3. Tercera pasada:
 - Compara 2 y 1 → intercambia → **[1, 2, 4, 5]**
 - Compara 2 y 4 → ok
 - Compara 4 y 5 → ok
4. Cuarta pasada:
 - No hay intercambios → ¡lista ordenada!

```
def bubble_sort_mejorado(lista):  
    n = len(lista)  
    for i in range(n):  
        intercambio = False  
  
        for j in range(0, n - 1 - i):  
            if lista[j] > lista[j + 1]:  
                lista[j], lista[j + 1] = lista[j + 1], lista[j]  
                intercambio = True  
  
        if not intercambio:  
            break
```

Python permite hacer asignaciones múltiples en una misma línea, y primero evalúa el lado derecho como una tupla (`lista[j + 1], lista[j]`), y luego desempaqueta esa tupla en el lado izquierdo, reasignando los valores.

Esto equivale a usar una variable temporal o auxiliar

```
temp = lista[j]  
lista[j] = lista[j + 1]  
lista[j + 1] = temp
```

¿Qué hace `n - 1 - i`?

En Bubble Sort, en cada pasada del algoritmo, el **mayor elemento restante "burbujea" hacia el final** de la lista. Entonces, en la primera pasada, se coloca el mayor al final; en la segunda pasada, el segundo mayor se acomoda antes de ese, y así sucesivamente.

Por lo tanto, **no es necesario seguir comparando hasta el final** en cada iteración, porque los últimos elementos ya están ordenados.

¿Qué hace exactamente Bubble Sort?

El Bubble Sort **compara pares de elementos adyacentes** y los intercambia si están en el orden incorrecto (por ejemplo, si el de la izquierda es mayor que el de la derecha).

En cada pasada por la lista:

- El **mayor valor "burbujea" al final**, porque se va intercambiando hacia la derecha en cada comparación.
- Entonces, **los valores más grandes se van acomodando al final** primero.

¿Qué pasa con los valores menores?

Sí, los **valores menores van quedando a la izquierda**, pero no porque los ignore, sino porque:

- En la primera vuelta, el más grande llega al final.
- En la segunda vuelta, el segundo más grande llega a la penúltima posición.
- Y así sucesivamente.

Los elementos más pequeños **también se siguen comparando**, pero sólo **hasta cierto punto**. Por eso el **range(0, n - 1 - i)**.

12.2 Selection Sort

Selection Sort (ordenamiento por selección) es un algoritmo de ordenamiento **simple pero eficaz para listas pequeñas**. A diferencia del Bubble Sort, **no intercambia constantemente elementos**, sino que **busca el mínimo (o máximo) en una parte de la lista y lo ubica en su posición definitiva**.

1. Se recorre toda la lista en busca del **valor mínimo**.
2. Una vez encontrado, **se intercambia con el valor de la primera posición** (si no es ya el menor).
3. Luego, se repite el proceso con el resto de la lista (desde la segunda posición en adelante), y así sucesivamente.

Paso a paso

Paso 1:

- Buscar el menor entre **[5, 3, 6, 2, 4]** → mínimo: **2**
- Intercambiar 2 con 5 → lista: **[2, 3, 6, 5, 4]**

Paso 2:

- Buscar el menor entre **[3, 6, 5, 4]** → mínimo: **3**
- Ya está en su lugar, no se intercambia

Paso 3:

- Buscar el menor entre **[6, 5, 4]** → mínimo: **4**
- Intercambiar 4 con 6 → lista: **[2, 3, 4, 5, 6]**

Y así sigue...

A tener en cuenta

- La lista se va dividiendo en dos partes:
Una parte **ordenada al principio** de la lista.
Una parte **desordenada que se va acortando**.

- Cada iteración **fija un valor en su lugar correcto**, por eso el **número de intercambios es mínimo** (a lo sumo uno por pasada).
- No es un algoritmo **estable** (es decir, puede cambiar el orden relativo de los elementos iguales).

```
def selection_sort(lista):  
    n = len(lista)  
    for i in range(n - 1):  
        indice_minimo = i  
  
        for j in range(i + 1, n):  
            if lista[j] < lista[indice_minimo]:  
                indice_minimo = j  
  
        if indice_minimo != i:  
            lista[i], lista[indice_minimo] = lista[indice_minimo], lista[i]
```

- **i** representa la posición actual donde queremos poner el siguiente menor.
- **indice_minimo** guarda la **posición del menor elemento encontrado** desde **i** hasta el final.
- El intercambio se hace una vez por iteración, **si realmente se necesita** (a diferencia del Bubble Sort, que puede intercambiar varias veces por pasada).

¿Por qué `range(n - 1)` en el primer `for`?

En el algoritmo de Selection Sort, **cada iteración fija un elemento en su posición final**.

- Si tenemos una lista de **n** elementos, **necesitamos hacer n - 1 pasadas** para ordenar todos.
- No hace falta revisar el último elemento con un bucle extra, porque si todos los demás están ordenados, **el último ya está en su lugar automáticamente**.

Ejemplo

Si ordenamos 5 elementos:

- La primera pasada pone el menor en la posición 0.
- La segunda pone el segundo menor en la posición 1.
- ...
- La cuarta pasada (índice 3) pone el cuarto menor en la posición 3.

- El que queda (índice 4) **ya es el mayor**, y no hace falta moverlo.

¿Por qué $\text{range}(i + 1, n)$ en el segundo for?

Ese bucle busca el menor **en la parte que todavía está desordenada**, es decir, **del índice siguiente a i hasta el final**.

- En cada pasada, asumimos que el mínimo está en la posición i .
- Entonces comparamos con los siguientes ($i + 1, i + 2, \dots, n - 1$) para ver si hay un valor menor.

Así evitamos comparar elementos que **ya están ordenados** y evitamos hacer trabajo innecesario.

12.3 - Insertion Sort (Ordenamiento por inserción)

El **Insertion Sort** es un algoritmo de ordenamiento simple, pero muy intuitivo. Su funcionamiento se parece a cómo una persona ordenaría manualmente un mazo de cartas: va tomando una carta a la vez y la inserta en el lugar correcto respecto a las ya ordenadas.

En otras palabras, **divide el vector en dos partes**:

- Una parte **ordenada** (al principio, solo el primer elemento).
- Una parte **no ordenada** (el resto del vector).

A medida que avanza el algoritmo, va sacando elementos de la parte no ordenada e **insertándolos en la posición correcta** dentro de la parte ordenada.

Paso a paso:

Lista original: [5, 3, 6, 2, 4]

Recorremos desde el segundo elemento.

Paso 1 (índice 1 – valor actual: 3)

- Comparar 3 con 5
- 3 es menor $\rightarrow$ mover 5 una posición a la derecha
- Insertar 3 en la primera posición
- **Lista actualizada:** [3, 5, 6, 2, 4]

Paso 2 (índice 2 – valor actual: 6)

- Comparar 6 con 5
- 6 es mayor $\rightarrow$ no se mueve

- Lista queda igual: [3, 5, 6, 2, 4]

Paso 3 (índice 3 – valor actual: 2)

- Comparar 2 con 6 → mover 6
- Comparar 2 con 5 → mover 5
- Comparar 2 con 3 → mover 3
- Insertar 2 en la primera posición
- Lista actualizada: [2, 3, 5, 6, 4]

Paso 4 (índice 4 – valor actual: 4)

- Comparar 4 con 6 → mover 6
- Comparar 4 con 5 → mover 5
- Comparar 4 con 3 → 4 es mayor → detener
- Insertar 4 en la posición correspondiente
- **Lista actualizada: [2, 3, 4, 5, 6]**

Resultado final ordenado: [2, 3, 4, 5, 6]

```
valores = [5, 3, 6, 2, 4]

for i in range(1, len(valores)):
    actual = valores[i]
    j = i - 1
    while j >= 0 and valores[j] > actual:
        valores[j + 1] = valores[j]
        j -= 1
    valores[j + 1] = actual

print("Lista ordenada:", valores)
```

Funcionamiento

1. Empieza desde el segundo elemento (índice 1).
2. Compara hacia atrás con los elementos anteriores.
3. Si el elemento actual es menor, se van desplazando los elementos mayores hacia la derecha.
4. Se inserta el valor actual en la posición correcta.
5. Se repite hasta el final del vector.

A tener en cuenta

- Es un algoritmo **in-place**: no usa estructuras auxiliares.
- Es **estable**: no altera el orden de elementos iguales.

12.4 - Merge Sort

Merge Sort es un algoritmo de ordenamiento que utiliza el enfoque de **divide y vencerás**. Esto significa que resuelve el problema dividiéndolo en partes más pequeñas, resolviendo esas partes de manera individual y luego combinándolas para obtener la solución completa.

A diferencia de métodos más intuitivos como el Insertion o el Selection Sort, Merge Sort es **recursivo** y muy eficiente, especialmente para listas largas.

El proceso de Merge Sort consta de tres etapas principales:

1. **Dividir (Divide):**
El vector se parte en dos mitades, una y otra vez, hasta que se obtienen sub-vectores de un solo elemento. Estos vectores, por definición, están ordenados.
2. **Conquistar (Sort/Conquer):**
Se ordenan recursivamente los sub-vectores, aunque en este caso el "ordenamiento" se da en el momento de fusionar.
3. **Fusionar (Merge):**
Se combinan los sub-vectores ordenados de manera que el resultado también esté ordenado. Esta es la parte más importante del algoritmo.

Ventajas:

- Muy eficiente para vectores grandes.
- Rendimiento consistente (siempre $O(n \log n)$).
- Estable.

Desventajas:

- Usa memoria adicional.
- Difícil de implementar sin recursividad o sin estructuras auxiliares.

Paso a paso

Supongamos que tenemos el siguiente vector:

[8, 4, 5, 2, 9, 1]

Paso 1: Dividir recursivamente

- **[8, 4, 5, 2, 9, 1]** → se divide en **[8, 4, 5]** y **[2, 9, 1]**
- **[8, 4, 5]** → se divide en **[8]** y **[4, 5]**
- **[4, 5]** → se divide en **[4]** y **[5]**

Paso 2: Fusionar sub-vectores ordenados

- Fusionar [4] y [5] → [4, 5]
- Fusionar [8] y [4, 5] → [4, 5, 8]
- [2, 9, 1] → se divide en [2] y [9, 1]
- [9, 1] → se divide en [9] y [1]
- Fusionar [9] y [1] → [1, 9]
- Fusionar [2] y [1, 9] → [1, 2, 9]

Paso 3: Fusionar los dos grandes bloques

- Fusionar [4, 5, 8] y [1, 2, 9] → [1, 2, 4, 5, 8, 9]

Funcionamiento

```
def merge_sort(vector):  
    if len(vector) <= 1:  
        return vector  
  
    medio = len(vector) // 2  
    izquierda = merge_sort(vector[:medio])  
    derecha = merge_sort(vector[medio:])  
  
    return fusionar(izquierda, derecha)
```

1. **if len(vector) <= 1:** Esta es la condición base de la recursión. Si el vector tiene 1 solo elemento (o está vacío), ya está ordenado, así que se devuelve tal cual.
2. **medio = len(vector) // 2** Calcula el **índice del medio** del vector. Se usa para dividir el vector en dos partes: izquierda y derecha.
3. Aplicar **recursivamente el mismo algoritmo** sobre ambas mitades del vector:
 - **vector[:medio]** es la mitad izquierda (del índice 0 hasta el anterior al medio).
 - **vector[medio:]** es la mitad derecha (del medio hasta el final).
 - Ambas mitades se ordenan **por separado**.
4. Una vez que ambas mitades están ordenadas, se las **fusiona en un solo vector ordenado** usando la función **fusionar**.

```
def fusionar(izq, der):  
    resultado = []  
    i = 0  
    j = 0  
  
    while i < len(izq) and j < len(der):  
        if izq[i] < der[j]:  
            resultado.append(izq[i])  
            i += 1  
        else:  
            resultado.append(der[j])  
            j += 1  
  
    resultado += izq[i:]  
    resultado += der[j:]  
    return resultado  
  
numeros = [8, 4, 5, 2, 9, 1]  
ordenado = merge_sort(numeros)  
print("Vector ordenado:", ordenado)
```

1. Inicialización:

- **resultado** es la lista vacía donde se irán guardando los elementos en orden.
- **i** y **j** son los índices que recorren las listas **izq** y **der**, respectivamente.

2. Comparación y mezcla:

- El **while** compara elementos de ambas listas mientras haya elementos en ambas.
- Se compara el elemento actual de cada lista (**izq[i]** y **der[j]**) y se agrega a **resultado** el menor de los dos.
- Se incrementa el índice de la lista desde la que se tomó el valor.

3. Agregar lo que quedó:

- Al terminar el **while**, puede que queden elementos en alguna de las listas.
- **resultado += izq[i:]** agrega lo que quedó de **izq**.
- **resultado += der[j:]** agrega lo que quedó de **der**.

4. Resultado final:

- La lista **resultado** ya contiene todos los elementos de **izq** y **der** en orden, por eso se devuelve.

Slicing

Para trabajar con ciertos algoritmos de ordenamiento o para manipular listas, es muy útil la herramienta llamada **slicing**. Esta permite tomar una porción de una lista, string o cualquier tipo de secuencia. Su sintaxis básica es **secuencia[inicio:fin:paso]**, y dependiendo de

cómo se completan esos valores (o si se omiten), se seleccionarán diferentes elementos. Por ejemplo, `lista[1:4]` devuelve los elementos desde la posición 1 hasta la 3 (el 4 no se incluye) y `lista[::-1]` devuelve la lista en orden inverso.

```
v = [10, 20, 30, 40, 50, 60]

print(v[1:4:1]) # [20, 30, 40] (desde el índice 1 hasta el 3, paso 1)
print(v[:2])   # [10, 20] (de 0 en 1, desde el inicio)
print(v[1:2])  # [20] (de 0 en 1, desde el índice 1)
print(v[::-1]) # [60, 50, 40, 30, 20, 10] (lista en orden inverso)
print(v[2:])   # [30, 40, 50, 60] (desde el índice 2 hasta el final)
print(v[:3])   # [10, 20, 30] (desde el inicio hasta el índice 2)
print(v[:])    # [10, 20, 30, 40, 50, 60] (toda la lista, copia)
```

12.5 Quick Sort

Quick Sort ("ordenamiento rápido") es un algoritmo de ordenamiento **divide y vencerás**, al igual que Merge Sort, pero funciona de manera diferente:

1. **Elige un pivote** (un elemento de la lista).
2. **Reordena** la lista de forma que:
 - Todos los elementos **menores al pivote** queden a su izquierda.
 - Todos los elementos **mayores al pivote** queden a su derecha.
3. **Repite** este proceso recursivamente sobre las sublistas (izquierda y derecha del pivote).

Este proceso se repite hasta que las sublistas tienen uno o ningún elemento, momento en el cual ya están ordenadas.

Paso a paso:

Supongamos que tenemos esta lista:

[7, 2, 1, 8, 6, 3]

Paso 1:

- Elegimos un pivote (por ejemplo, el último: 3)

Paso 2:

- Reordenamos para que todo menor que 3 quede a la izquierda y mayor a la derecha.

→ [2, 1] (pivote) [7, 8, 6]

→ Repetimos recursivamente sobre esas dos sublistas.

Funcionamiento

```
def quick_sort(vector):  
    if len(vector) <= 1:  
        return vector  
  
    pivote = vector[-1]  
    menores = [0] * (len(vector))  
    mayores = [0] * (len(vector))  
    i, j = 0, 0  
  
    for elemento in vector[:-1]:  
        if elemento < pivote:  
            menores[i] = elemento  
            i += 1  
        else:  
            mayores[j] = elemento  
            j += 1  
  
    return quick_sort(menores[:i]) + [pivote] + quick_sort(mayores[:j])
```

- **if len(vector) <= 1:**
Caso base: si el vector tiene 0 o 1 elemento, ya está ordenado.
- **pivote = vector[-1]**
Elegimos el último elemento como **pivote**.
- **menores = [] y mayores = []**
Creamos dos listas para separar los elementos.
- **for elemento in vector[:-1]:**
Recorremos todos **menos el pivote**.
- **if elemento < pivote:**
Si el elemento es menor al pivote, va a la lista **menores**, si no, a **mayores**.
- **return quick_sort(menores) + [pivote] + quick_sort(mayores)**
Ordenamos recursivamente y unimos: menores + pivote + mayores.